Embedded AI Project on FPGA using High Level Synthesis

Project group :

- **ABDERRAHMANE Mohamed Redha**
- **BAILLOT D'ESTIVAUX Téo**

Table of contents

1. Introduction and Problem Statement

Convolutional Neural Networks (CNNs) have become a reference solution for image classification tasks due to their ability to automatically extract hierarchical features from input data. However, CNN inference requires a large number of arithmetic operations, especially multiply-accumulate operations in convolution and fully connected layers. This computational cost poses significant challenges for embedded systems, where processing resources and power budgets are limited.

Field Programmable Gate Arrays (FPGAs) provide an attractive platform for CNN acceleration thanks to their inherent parallelism and configurability. By mapping compute-intensive operations onto dedicated hardware resources, FPGAs enable significant performance improvements compared to software execution on embedded processors.

This project focuses on the implementation of the LeNet convolutional neural network on a Xilinx Zynq-based FPGA platform. The objective is to evaluate and compare different execution strategies, including software execution on a PC, software execution on the embedded processor, and hardware execution using FPGA acceleration. The network is trained beforehand on the **MNIST** dataset using **TensorFlow** and **Keras**, and the trained weights are then deployed on the target platform. A detailed performance analysis is conducted to quantify the benefits of hardware acceleration.

2. Methodology

2.1 Global Design Flow and Tools

The project follows a structured design flow that progressively moves from software to hardware acceleration. A reference software implementation is first developed and validated on a PC platform. The same implementation is then ported to the Xilinx embedded platform to assess the impact of limited processing resources. Finally, the compute-intensive parts of the application are synthesized into hardware using **Vivado HLS**, enabling the exploitation of parallel hardware resources.

The design relies on fixed-point arithmetic in the hardware implementation to improve efficiency and resource utilization. We observed that a **14-bit fixed-point shift offset** provides an optimal compromise, keeping the results sufficiently close to the original floating-point values without noticeable accuracy loss. Fixed-point arithmetic is preferred on FPGA platforms because floating-point operations require more complex control logic and higher latency, while fixed-point computations rely on integer operations that map efficiently onto **DSP blocks**. This results in faster execution and reduced hardware resource consumption. The trained floating-point CNN weights are stored statically and embedded directly into the design; right before calling the `lenet_cnn` method, all weights are converted into fixed-point values.

2.2 LeNet CNN Architecture

The LeNet architecture implemented in this project follows the classical structure composed of convolutional layers, pooling layers, and fully connected layers. Convolutional layers extract spatial features from the input images, while pooling layers reduce dimensionality and improve robustness.The fully connected layers perform the final classification, followed by a softmax layer that converts the network outputs into normalized class probabilities. Using this architecture, the LeNet model achieves an accuracy of **97.82%** in floating-point and **97.79%** in fixed-point, corresponding to a **0.03%** accuracy difference between the two implementations.

2.3 Software Implementation

The software implementation is organized across multiple source files. The main inference flow is implemented in the main file, while the individual CNN layer functions are defined in `lenet_cnn_method_def.c`. Fixed-point arithmetic support is provided by `fixed_point.c`, and float trained weights are stored in `weight_tables.h`.
The CNN internal operations are implemented using nested loops that iterate over input channels and kernel dimensions of each layer, and the results are stored in the output channels.

2.3.1 Software Implementation

In the software-only version, all CNN computations are executed sequentially on the target processor. The nested loop structure closely follows the mathematical formulation of convolution and fully connected layers, iterating over channels, spatial dimensions, and kernel sizes. This implementation serves as a functional reference and ensures correctness of the LeNet inference pipeline before hardware acceleration. No explicit parallelism is introduced at this stage, and all operations are performed using standard control flow.

2.3.2 Hardware-Accelerated Implementation

For the hardware-accelerated version, the same C codebase is synthesized using Vivado HLS, and acceleration is achieved by inserting `#pragma HLS PIPELINE` directives in the most computationally dominant kernels, namely **Conv1**, **Conv2**, and **Fc1**. These three stages concentrate the majority of multiply-accumulate operations in the LeNet inference flow, making them the most impactful targets for improving hardware throughput.

In **Conv1**, the computation applies a **5×5 convolution kernel** to the input image in order to generate multiple output feature maps. This operation results in nested loops over the output spatial dimensions and the kernel dimensions (5×5), repeated for each output channel. In **Conv2**, the computational workload increases significantly, as the same **5×5 kernel** is applied across a larger depth, i.e., over multiple input feature maps, leading to nested loops over spatial positions, kernel dimensions, and input channels. The **Conv2 kernel contains 40 × 20 × 5 × 5 = 20,000 fixed-point coefficients**, corresponding to approximately **80 kB** of weight storage, making it one of the dominant contributors to both computation and memory usage. The **Fc1 layer performs dense matrix-vector multiplication**, where each output neuron accumulates contributions from a large input vector containing several hundred

features. The **Fc1 kernel is the largest in the network**, with **400 × 40 × 4 × 4 = 256,000 fixed-point coefficients**, corresponding to approximately **1 MB** of weight storage, and therefore dominates the overall computational workload.

Beyond loop-level pipelining, the hardware-accelerated implementation also optimizes memory usage by defining the **Conv2 kernel weights and Fc1 weight matrices as global variables**, rather than passing them as parameters to the `lenet_cnn` function. When large weight arrays are passed as function arguments, Vivado HLS may infer additional local storage, which can significantly increase BRAM utilization. By moving these kernels to global scope, the design avoids unnecessary array duplication, keeps BRAM usage under control, and ensures efficient access to large, read-only weight matrices within the hardware architecture.

By applying `#pragma HLS PIPELINE` to loops inside **Conv1**, **Conv2**, and **Fc1**, the hardware implementation is able to overlap successive loop iterations and increase computational throughput while preserving the functional behavior of the software reference model. This strategy focuses the available FPGA resources on the most computationally intensive parts of the network and directly contributes to the observed reduction in end-to-end execution time on the Xilinx platform.

3. Execution Platforms

To evaluate the different execution platforms, the inference phase of the LeNet network is performed on a test set of **10,000 handwritten digit images** from the **MNIST** dataset. The same trained model and test inputs are used across all platforms to ensure a fair and consistent performance comparison.

3.1 Software Execution on PC (SW-PC)

The reference implementation is executed on a PC platform using floating-point arithmetic. This execution benefits from a high-performance CPU, optimized compilers, and a large memory hierarchy. No architectural constraints are imposed, and the execution is purely sequential from the programmer's perspective.

The measured execution time for a full inference of the LeNet network on the PC is **20 seconds**, which represents the best achievable performance in this study. Given that the Xilinx platform is significantly more resource-constrained than a general-purpose PC, achieving an execution time of **400 seconds** on the embedded target is considered a satisfactory result in this context. In order to perform a meaningful performance comparison, both the software implementation and the hardware-accelerated implementation on the Xilinx platform are evaluated and analyzed.

3.2 Software Execution on Xilinx Platform (SW-Xilinx)

The same software implementation is compiled and executed on the embedded processor of the Xilinx Zynq-7000 platform. The algorithm remains unchanged, ensuring functional equivalence with the PC version.

Due to the lower clock frequency and limited computational resources of the embedded processor, execution time increases significantly. The measured inference time on the Xilinx platform in software-only mode is **900 seconds**. From a hardware synthesis perspective, the corresponding design exhibits a total latency of approximately **8.3 million clock cycles**, with moderate resource utilization, including **50 BRAM blocks**, **approximately 2,666 LUTs**, and **1,491 flip-flops**, representing a small fraction of the available FPGA resources. These results highlight the limitations of software-based CNN inference on embedded processors and motivate the use of hardware acceleration.

3.3 Hardware Execution on Xilinx Platform (HW-PAR)

To improve performance, the CNN inference is synthesized into hardware using Vivado HLS. The top-level hardware function corresponds to the complete `lenet_cnn` inference flow, including convolution, pooling, and fully connected layers.

In principle, the complete `lenet_cnn` inference function can be synthesized as a single hardware accelerator using Vivado HLS. In practice, passing large intermediate feature maps and weight arrays as function parameters led to implicit array duplication at function boundaries, which significantly increased memory usage, especially for **Conv2** and **Fc1** due to their large weight matrices. To avoid this overhead, large data structures were moved to **global variables** and most parameters were removed, keeping only the input to **Conv1** and the output of the final layer. Due to time constraints, the code structure was kept unchanged, and hardware acceleration was ultimately applied only to `Conv1`, which could be synthesized and integrated reliably.

The hardware design targets a clock period of **10 ns** and achieves an estimated clock period of **8.742 ns**, corresponding to an operating frequency of approximately **114 MHz**. The total estimated latency of the hardware accelerator is **990,659 clock cycles**.

Lower hardware accelerator latencies of approximately **940,000 to 950,000 clock cycles** were achievable; however, these configurations led to **near-saturation of LUT RAM resources**, approaching **100% utilization**. This left insufficient LUT RAM for the wrapper logic generated by Vivado SDx to deploy the accelerator on the target board. Consequently, these configurations were not selected for the final implementation.

Hardware resources are effectively utilized, with strong usage of DSP blocks for arithmetic operations and BRAMs for data storage. The design exploits hardware parallelism to accelerate the most computationally intensive operations.

The measured execution time for the hardware-accelerated implementation on the Xilinx platform is **450 seconds**, demonstrating a clear improvement over the software-only execution on the same platform.

However, the improvement remains limited compared to the theoretical potential of FPGA acceleration. This is explained by the fact that only **Conv1** is effectively accelerated in hardware, while the remaining layers continue to execute in software on the embedded processor. As a result, the overall execution time is still largely influenced by software execution of deeper layers, especially Conv2 and Fc1, which are known to dominate the

computational workload.
In theory, an execution time of approximately **350 seconds** could be achieved if the entire `lenet_cnn` inference flow were synthesized into hardware. However, due to time constraints, this full hardware integration could not be realized within the scope of this project.

4. Performance Analysis

4.1 Execution Time Comparison

The measured execution times for the different implementations are summarized as follows:

Execution Platform	Implementation Type	Execution Time (s)	Images/s
PC	Software	20	500.0
Xilinx Platform	Software	900	11.1
Xilinx Platform	Hardware (Parallelized)	450	22.2

4.2 Analysis and Interpretation

The measured execution times clearly illustrate the impact of hardware acceleration on the Xilinx platform. Software execution on the embedded processor exhibits a very high inference time of **900 seconds** reflecting the limitations of executing compute-intensive CNN workloads on a resource-constrained processor. In contrast, the hardware-accelerated implementation significantly improves performance by exploiting FPGA parallelism.

The hardware-accelerated version reduces the inference time to **450 seconds**, corresponding to an improvement of approximately **50%** compared to the software-only execution on the same platform. This substantial reduction confirms that offloading the most computationally intensive CNN operations to dedicated hardware is an effective strategy for accelerating inference in embedded systems.

On the Xilinx platform, software execution achieves a throughput of **11.1 images/s**, while hardware acceleration increases this to **22.2 images/s**, corresponding to a **2×** improvement.

4.3 Resource Utilization

The hardware implementation makes efficient use of FPGA resources. DSP blocks are heavily utilized to accelerate arithmetic operations, while BRAMs store intermediate feature maps and network weights. Logic utilization remains moderate, leaving room for future extensions or additional parallelism.

The table below summarizes the latency and resource utilization for both the software and hardware implementations on the Xilinx platform.

Implementation	Latency (cycles)	BRAM (18 Kb)	DSP48E	LUT
Software Implementation	8,300,845	50 (17%)	12 (5%)	3,182 (5%)
Hardware-Accelerated Implementation	990,659	82 (29%)	202 (91%)	13,785 (25%)

The hardware-accelerated implementation reduces latency by nearly an order of magnitude compared to the software version, at the cost of higher DSP and LUT utilization. This trade-off highlights the effectiveness of FPGA parallelism for accelerating CNN inference while keeping overall resource usage within acceptable limits.

5. Conclusion

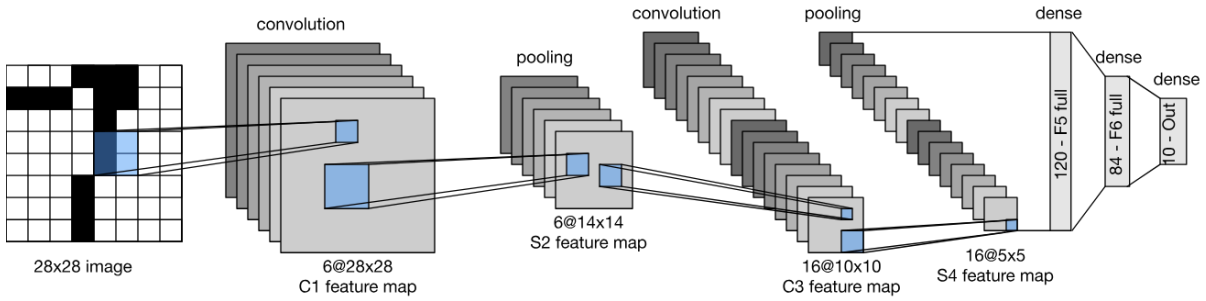
The experimental results demonstrate that hardware acceleration on the Xilinx Zynq platform provides a meaningful performance improvement over software execution on the embedded processor. While the software-only implementation exhibits a high inference time of **900 seconds**, the hardware-accelerated version reduces this time to **450 seconds**, corresponding to a **50% performance improvement**. This clearly shows that FPGA parallelism can effectively mitigate the computational limitations of embedded processors for CNN inference.

The results also highlight the cost of this performance gain in terms of hardware resources. The reduction in latency is achieved through increased utilization of DSPs and LUTs, while BRAM usage remains within acceptable limits. Attempts to further reduce latency revealed a practical integration limit, as more aggressive configurations led to near-saturation of LUT RAM resources, preventing successful deployment on the target board. This confirms that achievable performance is tightly constrained by available FPGA resources and system-level integration requirements.

Overall, the results underline the importance of balancing latency reduction and resource utilization. Even partial hardware acceleration, limited to selected layers, delivers substantial benefits and represents a viable compromise for deploying CNN inference on resource-constrained FPGA-based embedded systems.

Appendix

LeNet network



The **LeNet** network takes as input a 28×28 grayscale image from the MNIST dataset. This input layer does not perform any computation and simply forwards the pixel values to the first processing stage

The first convolutional layer applies a set of small convolution kernels, typically of size 5×5, to the input image. This layer is responsible for extracting low-level visual features such as edges, corners, and simple shapes. A non-linear activation function is applied after the convolution to introduce non-linearity into the model.

The first pooling layer performs spatial downsampling, usually with a 2×2 window. This reduces the spatial resolution of the feature maps while preserving the most relevant information. Pooling helps decrease computational complexity and improves robustness to small translations in the input image.

The second convolutional layer operates on the pooled feature maps and applies a larger number of filters than the first convolutional layer. It extracts higher-level features by combining the simpler patterns detected earlier, such as parts of digits or more complex shapes.

The second pooling layer further reduces the spatial dimensions of the feature maps. This step prepares the data for the fully connected part of the network and limits the number of parameters, which helps control overfitting and computational cost.

The first fully connected layer takes the flattened feature maps as input and learns global feature combinations across the entire image. This layer plays a key role in transforming the extracted features into a representation suitable for classification.

The final fully connected layer produces the output scores corresponding to the ten digit classes. A softmax function is typically applied to convert these scores into probabilities, allowing the network to predict the most likely digit class.